

claude-windows / accd47af-95a3-46a7-9b51-b3eb5066a777

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\accd47af-95a3-46a7-9b51-b3eb5066a777\subagents\workflows\wf_4fd1a4ea-6c0\agent-ad21592e30d70e932.jsonl`  
- SHA-256: `f93c0a528aad812afc3efb8909f00fb7192867c187141c8de73e46b7477f7c32`  
- Source modified: `2026-07-02T09:33:04+00:00`  
- Imported at: `2026-07-05T16:48:24+00:00`  
- project: `wf_4fd1a4ea-6c0`  
- session_id: `accd47af-95a3-46a7-9b51-b3eb5066a777`
```

Transcript

1. user (2026-07-02T09:30:45.754Z)

ADVERSARIAL VERIFIER. A reviewer of the P23 main.py->orchestration.py extraction claims:

TITLE: Delta enumeration in the PR claim is stale: 4 unenumerated (but necessary and correct) deltas exist in orchestration.py
SEVERITY: major
DETAIL: The review request claims the 10 members are AST-identical to master except 3 enumerated deltas. The worktree file (modified 14:54 IST, mid-review) contains FOUR additional intentional deltas: (a) `_execute_processing` resolves `compute_video_id` via `_main.compute_video_id` (orchestration.py:386) instead of a bare call; (b) it resolves `emit_indexer_report` via `_main.emit_indexer_report` (orchestration.py:571); (c) `_execute_compile` gained its own call-time `'import backend.main as _main'` (orchestration.py:679); (d) it constructs the compiler via `_main.VideoCompiler` (orchestration.py:702). Correspondingly, `VideoCompiler / compute_video_id / emit_indexer_report` are NOT imported at orchestration's top. These deltas are exactly what keeps the existing test seams alive ? monkeypatch.setattr(m, "VideoCompiler") at tests/test_api_endpoints.py:411,455,486,531,575, setattr(m, "emit_indexer_report") at :782, and setattr(m, "compute_video_id") at :817 all target names consumed inside the moved functions; with bare calls (the file's earlier state this session) those 7+ tests would not intercept and several would fail. I confirmed empirically the loaded function globals contain no `VideoCompiler/compute_video_id/emit_indexer_report` and that the affected tests pass against the current file. ACTION: re-run the machine AST-identity verification against the FINAL file and update the PR description / delta enumeration to list all 7 deltas (3 original + these 4) before commit ? the file churned under review, so pin the verified state.
EVIDENCE: wt-docs/backend/orchestration.py:386, wt-docs/backend/orchestration.py:571, wt-docs/backend/orchestration.py:679, wt-docs/backend/orchestration.py:702; consumers: wt-docs/tests/test_api_endpoints.py:411,455,486,531,575,782,817

REFACTOR UNDER REVIEW (uncommitted, in worktree C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/accd47af-95a3-46a7-9b51-b3eb5066a777/scratchpad/wt-docs, branch refactor/p23-orchestration-split off origin/master):

```
- NEW backend/orchestration.py (739 lines): 10 members moved VERBATIM from backend/main.py:  
  _finalize_reingest, _report_config_for_input, _ingest_remote_segments, _cloud_available,  
  _maybe_dispatch_remote, _execute_processing, _remote_input_size_bytes, _CompileError,  
  _resolve_compile, _execute_compile.  
- backend/main.py 2360 -> 1703 lines; re-exports all 10 names via "from backend.orchestration  
import (...)" (noqa F401);  
  5 now-unused imports removed (csv, math, Tuple, StitchingConfig, localize_watermark).  
- Enumerated intentional deltas (everything else is AST-identical, machine-verified):  
  (1) _maybe_dispatch_remote: "req: ProcessRequest" -> string forward-ref "ProcessRequest"  
  (TYPE_CHECKING import).  
  (2) _execute_processing: inserted "import backend.main as _main" after docstring;  
  global_config/db_instance/  
    ProcessRequest resolved via _main.<name> at call time (the job_worker seam convention).
```

(3) two "# type: ignore[...]" comments (AST-invisible).

- NEW tests/test_main_module_budget.py: line budget 1750, route ceiling 18, re-export identity check.
- Verified already: AST-identity (10 moved + 51 retained members), ruff+format clean, mypy green (142 files),

ratchet test passes. Full suite running separately.

Inspect with: `git -C "C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/accd47af-95a3-46a7-9b51-b3eb5066a777/scratchpad/wt-docs" diff` (and `git -C "C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-Projects-badminton-highlight-indexer/accd47af-95a3-46a7-9b51-b3eb5066a777/scratchpad/wt-docs" status`); the new files are untracked/modified in the worktree.

READ-ONLY: do not modify the worktree, do not checkout/commit.

Independently verify in the worktree (READ-ONLY). Default skeptical: `real=true` ONLY if you can reproduce the evidence AND the failure scenario is concrete (a test that would fail, a runtime path that breaks, an operational regression). A theoretical concern with no concrete consumer = `real=false`.

2. user (2026-07-02T09:30:54.262Z)

```
M backend/main.py
?? backend/orchestration.py
?? tests/test_main_module_budget.py
refactor/p23-orchestration-split
-rw-r--r-- 1 avidu 197609 34320 Jul  2 14:54 backend/orchestration.py
```

3. user (2026-07-02T09:31:02.543Z)

```
# consumers (hash / validators / segmenter) use the resolved local path.
local_video = ""
try:
    local_video = storage.acquire_local_input(
        req.video_path, getattr(config, "storage", None)
    )
    video_id = _main.compute_video_id(local_video)
except Exception:
    storage.cleanup_acquired_local_input(
        req.video_path, local_video, getattr(config, "storage", None)
    )
    raise
try:
    (response or {}).get("status") if isinstance(response, dict) else None,
)
# Always emit the per-video observability report (next to the video, or to
# report.output_dir). Never raises. Surface the paths in the response. (The compute-path
# proof lives on the job result + the [COMPUTE] log, not the report ? obsreport's
record_run
# doesn't surface arbitrary run signals.)
paths = _main.emit_indexer_report(
    db=db,
    video_id=video_id,
    # Resolved LOCAL path, not the raw URI: obsreport writes the report next to the
    # video via os.path.abspath(video_path), and a remote URI (gs://, gcs://, local://)
    """Worker handler for a ``kind='compile'`` job (#329): re-resolve the segments, stitch the
reel,
and return the {status, filepath, download_url} result the SPA polls for. Returns a
``status='failed'`` result for an EXPECTED problem (missing video / no segments / a stitch
error)
so the SPA always gets a clean verdict instead of a worker-crash error."""
import backend.main as _main # call-time seam: VideoCompiler stays patchable on
backend.main

notify = progress or (lambda stage: None)
params = json.loads(job.get("params") or "{}")
video_id = job.get("video_id") or ""
```

```

segment_ids = params.get("segment_ids") or []
locale = params.get("locale")

notify("resolving")
try:
    video, kept, out_name = _resolve_compile(
        db, cfg, video_id, segment_ids, params.get("output_filename", "")
    )
except _CompileError as e:
    return {"status": "failed", "message": e.detail, "video_id": video_id}

stitching = getattr(cfg, "stitching", None) or StitchingConfig()
output_dir = (
    getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
)
# Localize the branding watermark to the UI locale the SPA rendered in (recorded on the
job);
# unknown/absent locale keeps the configured default unchanged.
localized_watermark = localize_watermark(stitching.watermark, locale)
compiler = _main.VideoCompiler(
    output_dir,
    begin_padding=stitching.begin_padding,
    end_padding=stitching.end_padding,
    watermark=localized_watermark,
744 backend/orchestration.py

```

4. assistant (2026-07-02T09:31:10.068Z)

The claimed lines exist (offsets within a couple lines). Now check top-of-file imports and the test seams.

5. user (2026-07-02T09:31:15.380Z)

```

16: ``ProcessRequest``, ``compute_video_id``, ``emit_indexer_report``, ``VideoCompiler``.
386:     video_id = _main.compute_video_id(local_video)
571:     paths = _main.emit_indexer_report(
679:     import backend.main as _main # call-time seam: VideoCompiler stays patchable on
backend.main
702:     compiler = _main.VideoCompiler(

```

6. user (2026-07-02T09:31:16.512Z)

""""Processing / compile orchestration, extracted from ``backend.main`` (P23; re-cut of #390).

The full hash -> validate -> segment -> report pipeline (``\_execute\_processing``), the Cloud Run dispatch + ingest path (``\_maybe\_dispatch\_remote`` / ``\_ingest\_remote\_segments``), and the shared compile resolution + worker handler (``\_resolve\_compile`` / ``\_execute\_compile``).

Extracted verbatim from ``backend.main`` to shrink the orchestrator; behavior is unchanged.

SEAM DISCIPLINE (same convention as ``backend.api.job\_worker``): anything that must remain monkeypatchable-on-``backend.main`` or that lives in ``backend.main`` (the transitional ``db\_instance``/``global\_config`` globals and the ``ProcessRequest`` model) is resolved through the ``backend.main`` module object at call time (``import backend.main as \_main``), never as a bare local. ``backend.main`` re-exports every name below, so ``monkeypatch.setattr(backend.main, ...)`` and ``import backend.main as m; m.\_execute\_processing(...)`` keep working unchanged. This module must NOT import ``backend.main`` at module level (main imports us for the re-export).

Routed seams (rebind-patched on ``backend.main`` by tests): ``global\_config``, ``db\_instance``, ``ProcessRequest``, ``compute\_video\_id``, ``emit\_indexer\_report``, ``VideoCompiler``. Object-attribute patches (``m.storage.fetch``, ``m.segmenter\_registry.get``, ...) hit the shared objects and need no routing.

""""

```

import csv
import json
import logging
import math
import os
import tempfile
from typing import TYPE_CHECKING, Any, Dict, List, Optional, Tuple
from urllib.parse import quote

from backend import storage
from backend.config import AppConfig, StitchingConfig
from backend.pipeline.segmenters.base import segmenter_registry
from backend.pipeline.stitcher.watermark_i18n import localize_watermark
from backend.pipeline.validators.base import registry
from backend.results import Failure
from backend.utils.paths import output_base, safe_output_filename

if (
    TYPE_CHECKING
): # runtime import would be circular (main imports us for the re-export)
    from backend.main import ProcessRequest # noqa: F401

logger = logging.getLogger(__name__)

def _finalize_reingest(db, video_id: str, segments, play_wm: int, cand_wm: int) -> None:
    """Commit a (re)segmentation with destroy-AFTER-confirm semantics.

    On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
    fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark) and
    keep the prior good results intact ? a failed/empty re-run never destroys prior data.
    """
    if segments:
        db.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
    else:
        db.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)

def _report_config_for_input(config: Any, raw_input: str) -> Any:
    """Return report config that keeps remote-input reports out of disposable scratch."""

```

7. user (2026-07-02T09:31:27.500Z)

```

411: mp setattr(m, "VideoCompiler", _FakeCompiler)
455: mp setattr(m, "VideoCompiler", _CaptureCompiler)
486: mp setattr(m, "VideoCompiler", _fake_ok_compiler())
531: mp setattr(m, "VideoCompiler", _BoomCompiler)
575: mp setattr(m, "VideoCompiler", _FakeCompiler)
782: mp setattr(m, "emit_indexer_report", fake_emit)
817: mp setattr(m, "compute_video_id", fail_hash)
4: import os
5: import subprocess
6: import sys
7: from unittest.mock import MagicMock, patch
9: import pytest
12: from fastapi.testclient import TestClient
14: import backend.main as m
15: from backend.config.models import AppConfig
16: from backend.infrastructure.database import Database
17: from backend.pipeline.validators.base import ValidationResult

```

8. assistant (2026-07-02T09:31:37.640Z)

All 7 test seams confirmed. Now verify these are genuine deltas versus master ? check what master's main.py does at those call sites.

9. user (2026-07-02T09:31:41.877Z)

```
1230: def _execute_processing(
1271:     video_id = compute_video_id(local_video)
1456:     paths = emit_indexer_report(
1790: def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
1816:     compiler = VideoCompiler(
2094:     compiler = VideoCompiler(
2216:         video_id = compute_video_id(local_video)
2278:         paths = emit_indexer_report(
```

10. user (2026-07-02T09:32:02.736Z)

```
module: backend.orchestration
VideoCompiler in globals: False
compute_video_id in globals: False
emit_indexer_report in globals: False
re-export identity: True True
```

11. user (2026-07-02T09:32:38.228Z)

```
C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\google\auth\_default.py:113: UserWarning: Your application has authenticated using end
user credentials from Google Cloud SDK without a quota project. You might receive a "quota
exceeded" or "API not enabled" error. See the following page for troubleshooting:
https://cloud.google.com/docs/authentication/adc-troubleshooting/user-creds.
warnings.warn(_CLOUD_SDK_CREDENTIALS_WARNING)
```

```
-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
51 passed, 1 skipped, 4 warnings in 25.64s
```

12. assistant (2026-07-02T09:32:53.187Z)

All evidence reproduced. Reporting the verdict.

13. user (2026-07-02T09:33:04.590Z)

Structured output provided successfully